

GAP FILLING METHOD (Interval DP Method)

Instead of filling row by row, we fill according to substring length.

Idea

First solve:

Length = 1

Then:

Length = 2

Then:

Length = 3

and so on.

Because:

```
dp[i][j]
```

depends on smaller substrings.

Gap Meaning

```
gap = j - i
```

Example

```
gap = 0
```

```
a  
b  
c  
d
```

Single characters.

```
gap = 1
```

```
ab  
bc  
cd
```

Substrings of length 2.

```
gap = 2
```

```
abc  
bcd
```

Substrings of length 3.

Code

```
def longestPalindromeSubseq(s):  
    n = len(s)  
  
    dp = [[0]*n for _ in range(n)]  
  
    for gap in range(n):  
  
        for i in range(n-gap):  
  
            j = i + gap  
  
            if gap == 0:  
                dp[i][j] = 1  
  
            elif s[i] == s[j]:  
                dp[i][j] = 2 + dp[i+1][j-1]
```

```
        else:
            dp[i][j] = max(
                dp[i+1][j],
                dp[i][j-1]
            )

    return dp[0][n-1]
```

Another Length-Based Version

```
for length in range(2,n+1):
    for i in range(n-length+1):

        j = i + length - 1
```

This is equivalent to the gap method.

When Should I Think of Gap Filling?

Whenever the state is:

```
f(i,j)
```

and dependencies are:

```
f(i+1,j)
f(i,j-1)
f(i+1,j-1)
```

or generally:

```
f(i,k)
f(k+1,j)
```

These are Interval DP problems.

Problems Using Gap Filling

1. Longest Palindromic Subsequence
 2. Minimum Insertions to Make Palindrome
 3. Palindrome Partitioning
 4. Matrix Chain Multiplication
 5. Burst Balloons
 6. Minimum Cost to Cut a Stick
 7. Count Palindromic Subsequences
 8. Boolean Parenthesization
-